

실시간 채팅 서비스

HTTP · WebSocket 혼합 환경의 상태 정합성 해결

- Spring Boot Monolith
- WebSocket
- 인메모리 상태
- Semaphore 동시성
- TTL 세션 관리

WHY 프로젝트 선택 배경

REST API(Stateless)와 WebSocket(Stateful)이 한 서비스에서 공존할 때 발생하는 상태 정합성 문제를, 프레임워크에 기대지 않고 직접 설계해 해결하기 위해 시작한 프로젝트.

WHAT 핵심 문제

문제 1 · Race Condition

동시 입장 시 정원 초과

정원 확인과 입장 처리 사이를 다른 요청이 비집고 들어와 정원을 넘어선다.

문제 2 · Session Leak

비정상 종료 시 자원 미회수

탭·브라우저 종료와 새로고침이 같은 신호로 들어와 유령 세션이 쌓인다.

GOAL 목표 · 성과

목표 1 · 데이터 정합성

정원 초과 0건

방별 Semaphore로 정원을 원자적으로 통제 — 동시 접속에서도 초과 0건.

목표 2 · 자원 효율성

유령 세션 자동 정리

서버 권위 sessionKey와 TTL 유예로 미접속 세션을 자동 회수한다.

ChatService 실시간 채팅 시스템 아키텍처

HTTP·WebSocket 이원 · 인메모리 상태 · Spring Boot Monolith



방 입장 프로토콜

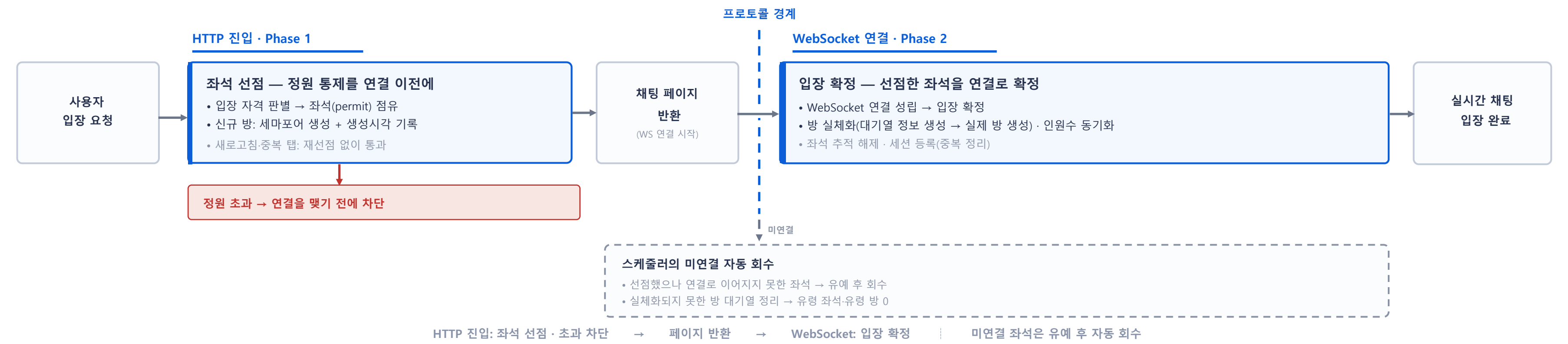
문제 정의 입장은 HTTP 요청으로 시작해 WebSocket 연결로 진행되어 하나의 입장이 서로 다른 두 프로토콜로 진행되어 서로 다른 프로토콜 간 정합성 문제를 해결한다.

의사결정 지점 정원이라는 비즈니스 규칙의 통제를 어느 프로토콜 단계에 둘 것인가.



후보 비교

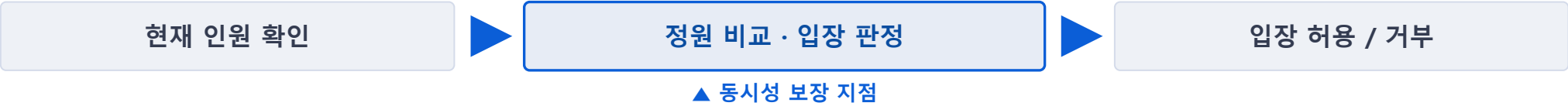
후보	정원 점유 시점	정원 초과 차단
① 단일 단계 — 연결 시 점유·확정	WebSocket 연결 시점	연결을 맺은 뒤 차단은 불필요한 비용 발생
② 2단계 — 진입 선점 + 연결 확정 ✓ 채택	HTTP 진입 시점 → 연결로 확정	연결 이전에 차단



방 정원 동시성 제어

문제 정의 정원 5인 방에 여러 사용자가 **같은 순간** 입장하면, '인원 확인과 입장 허용' 사이를 다른 요청에 의해 동시성 문제가 발생하여 **정원을 초과한다**.

의사결정 지점 입장 자격 판별의 **check-then-act**(인원 확인 → 입장 허용) 구간을 어떤 동시성 메커니즘으로 원자화할 것인가.



① 경합 상황 — **check-then-act**의 비원자성

잔여 1석인 순간, 세 요청이 '확인'과 '입장' 사이에 서로 끼어든다

A

B

C

확인 (Check)

잔여 1석 = true

잔여 1석 = true

잔여 1석 = true

입장 (Act)

입장 (+1) 실행

입장 (+1) 실행

입장 (+1) 실행

결과

정원 5 → 6명 입장

원자성 부재 = 정원 초과

② 후보 4종 — **check-then-act** 원자화 방법

✓ 채택 Semaphore (lock-free)

메커니즘	원자성	임계구역 점유	정원 초과
IF-ELSE	✗	없음	발생
synchronized	✓	전체 직렬화	없음
ReentrantLock	✓	전체 직렬화	없음
Semaphore.tryAcquire	✓ (CAS)	없음 (lock-free)	없음

③ 안전성 4규칙 측정 · 상호 배제 3기법 (오류 탐지 기준)

기법	값 불일치	경합	정원 초과	상태 전이
IF-ELSE	127 (6.35%)	1,839 (92%)	1건 (0.05%)	183 (9.15%)
Synchronized	0	0	0	0
ReentrantLock	0	0	0	0

→ 비즈니스 규칙(정원 초과) 위반은 **IF-ELSE 1건**뿐 · Synchronized·ReentrantLock은 전 규칙 0

④ Semaphore — 3단계 계층적 일관성 · 4규칙을 3지표로 재정의

단계	측정 지표	실측	의미
비즈니스 규칙	정원 초과 (R3)	0건 (0%)	락과 동등
동시 실행	경합→동시 실행 (R2)	974건 (48.7%)	의도된 병렬 (lock-free)
내부 일관성	순차 일관성 (R1+R4)	197건 (9.85%)	부분 보장 의도된 트레이드오프

→ 락과 동등한 지점은 '정원 초과 0' 하나 · 나머지는 결함이 아니라 **의도된 동시성**

결정 Semaphore

근거 1 · 비즈니스 안전성 동등

정원 초과 **0건(0%)**으로 Synchronized·ReentrantLock과 동일. IF-ELSE만 실측 1건으로 탈락.

근거 2 · 충돌 범위 일치 · lock-free

잠그지 않고 permit으로 '정원 경계'만 통제. 거부 처리에서 ReentrantLock 대비 **4.4배 빠름**.

근거 3 · 결정 사유 (방법론적 기여)

'비즈니스 정합성(정원 0)'과 '순차 일관성(9.85% 차이)'을 분리·재정의하여 정원 규칙엔 순차 일관성 불필요,

구현 위치 concurrency/SemaphoreRegistry (tryAcquire · getAvailablePermits) · joinroom/RoomJoinService.confirmJoinRoom | 측정 200 동시 요청 × 10개 방 = 2,000건 · System.nanoTime() 나노초

세션 생명주기 통제

문제 정의 브라우저의 WebSocket 종료 이벤트는 최초·새로고침·중복 탭·비정상 접근을, 그리고 ‘잠깐 떠남’과 ‘영구 이탈’을 같은 종료 신호 하나로만 전달한다.

의사결정 지점 ① 접속 유형의 판정 기준을 클라이언트가 아니라 서버가 소유(sessionKey) · ② 비명시 종료 자원을 즉시가 아니라 일정 시간 유예 후 회수

생명주기 흐름



① 접속 유형 식별 — 서버 권위 sessionKey

serverKey·clientKey의 존재·일치 조합으로 분기 (평가 순서 = 로직)

serverKey	clientKey	두 키 일치	판정	처리
없음	없음	—	최초 입장	서버가 신규 세션키 발급 후 세션 등록
없음	있음	—	stale (조작/오류)	서버 기준으로 강제 초기화 후 최초 입장
있음	있음	일치	동일 탭 새로고침	유예 상태 정리 후 동일 세션으로 복귀
있음	없음	—	중복 탭	기존 세션 강제 종료 후 새 세션 교체
있음	있음	불일치	동기화 불일치	비정상 접근으로 보고 연결 차단

판정 기준(sessionKey)을 서버가 발급·소유·검증하므로 클라이언트 조작·오류를 무력화한다. 클라이언트는 서버가 내려준 키를 받아 보관·동기화만 한다.

② 비명시 종료 자원 회수 — TTL 유예

즉시 회수하면 새로고침 복귀가 깨진다 — 유예 후 재접속이 없을 때만 회수

회수 시점 전략 — 3후보 비교

후보	새로고침 복귀	유령 세션
즉시 회수	X 복귀 깨짐	없음
회수 보류	✓ 복귀	X 무한 누적
TTL 유예 후 판정 (채택)	✓ 복귀	없음 (만료 시 회수)

시점별 동작과 핵심 맥락

시점	동작	핵심 맥락 (왜)
비명시 종료	임시 보류 상태로 표시 · 브로드캐스트 제외 · 방은 유지	즉시 회수하면 새로고침 복귀가 깨짐 → 판단 보류
유예 시간 내 재접속	보류 해제 후 동일 세션으로 복귀	유예 창 안의 재접속은 일시 단절로 인정
유예 만료	좌석 · 세션 · 인원수 일괄 회수	미접속 지속 = 영구 이탈 확정 → 유령 세션 0

결정 판정 기준은 서버가 소유(sessionKey) · 회수는 시간 기반(TTL 유예) — 클라이언트 신호 없이 서버가 세션 생명주기를 통제

근거 1 · 시간 기반 사후 판정 (회수)

종료와 새로고침이 같은 신호로 들어오므로, 유예 시간을 두고 재접속 유무로 영구 이탈을 구분 한다.

근거 2 · 결정 사유 (생명주기 일관)

모든 접속 경우를 다섯 유형으로 분류하며 관리한다.